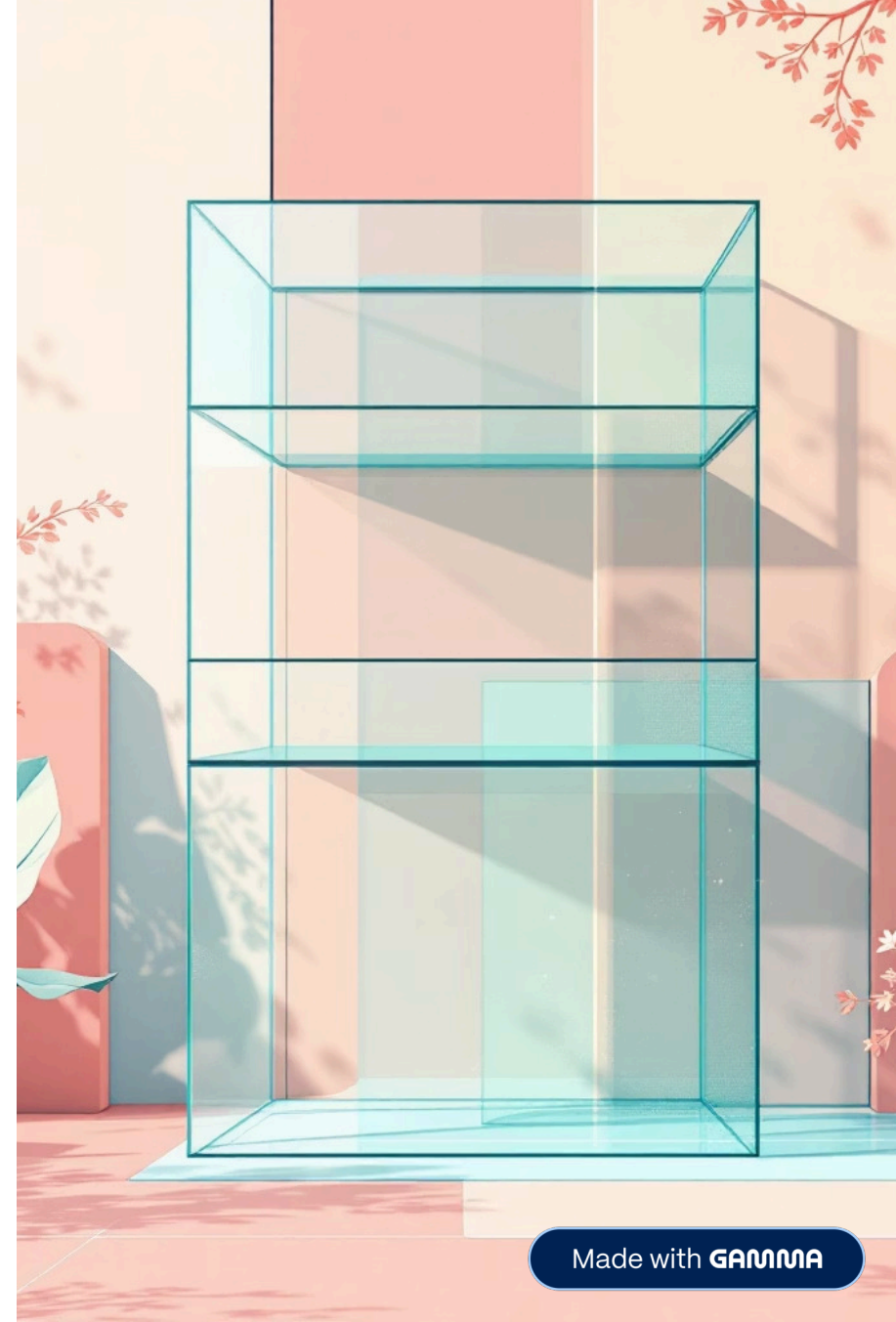


CSC207 — DATABASE SYSTEMS

F24 MULTIMEDIA & GAMING

Three Levels of Database Architecture

Instructor: Engr. Arifa



Why Three Levels?

The **ANSI/SPARC architecture model** was introduced to solve a fundamental problem: how do we let many users interact with one database without chaos? The answer is **separation of concerns** across three distinct levels of abstraction.



Data Security

Users only access what they need



Data Abstraction

Complexity hidden behind clean interfaces



Data Independence

Changes at one level don't break others

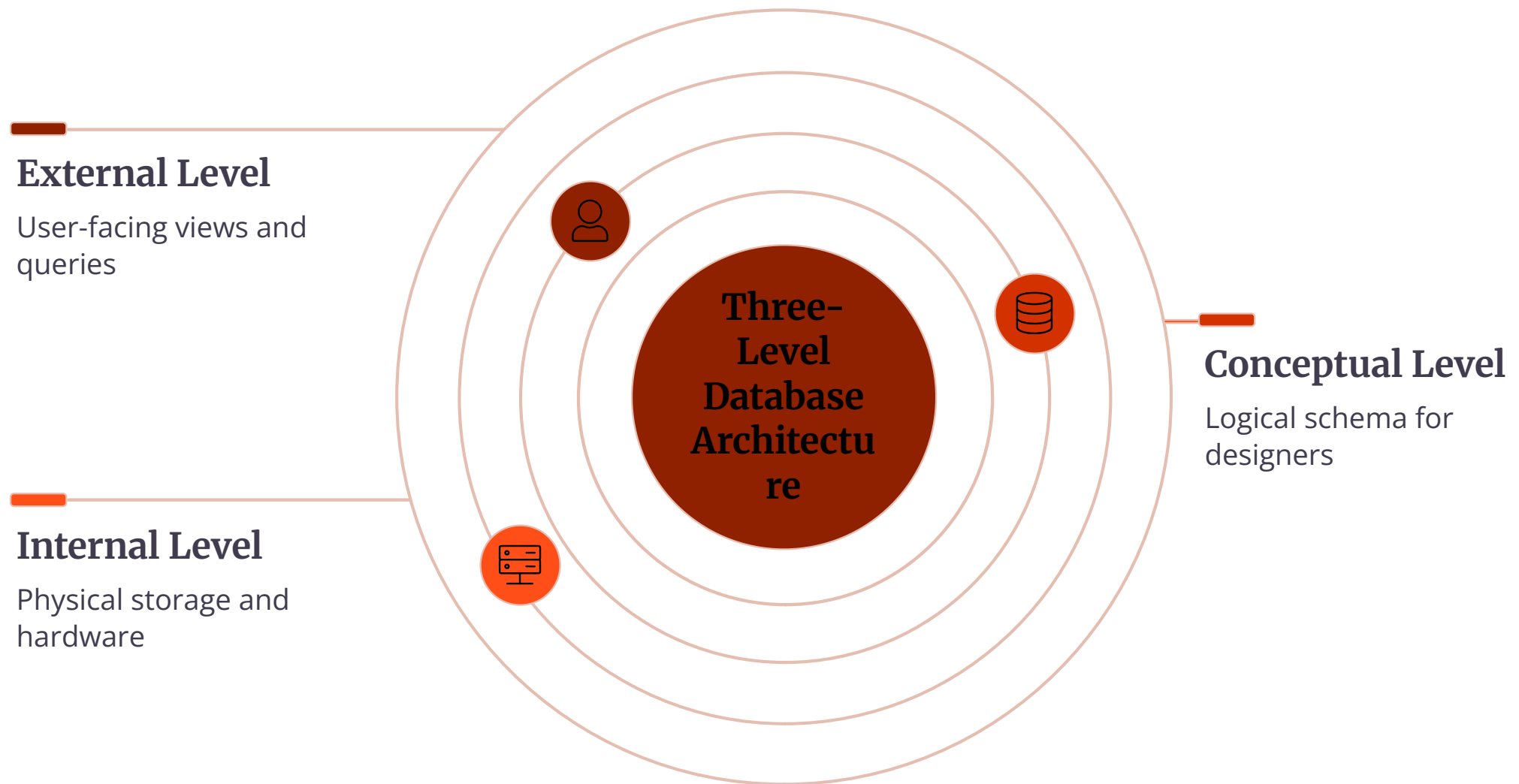


Efficient Management

DBAs work at the right level of detail

The Three Levels at a Glance

Each level serves a distinct audience and purpose. Together, they form a coherent framework for organising every aspect of a database system.



The external level sits closest to the user, the internal level closest to the hardware, and the conceptual level bridges the two — acting as the single source of logical truth for the entire organisation.

External Level (View Level)

The **External Level** is the highest level of abstraction. It describes only a specific portion of the database as seen by a particular user or application. Each stakeholder interacts with a **customised view** — never the raw data.



HR View

Employee salary & benefits details

Sales View

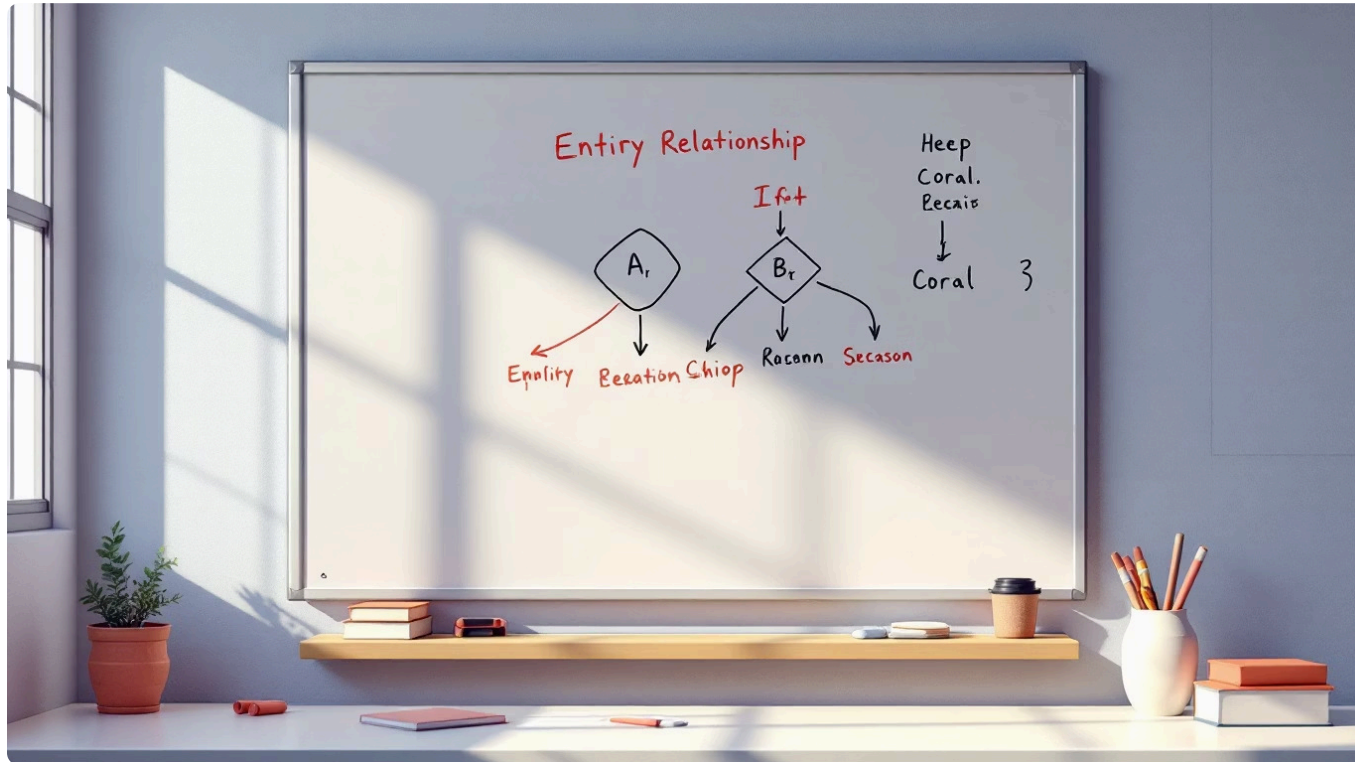
Customer orders & revenue data

Student View

Personal grades & enrolment only

📌 **✓ Data security** — sensitive data stays hidden · **✓ Customisation** — each role sees relevant info · **✓ Simplified interaction** — no SQL expertise required

Conceptual Level (Logical Level)



The **Conceptual Level** describes the complete logical structure of the entire database. It defines **what** data is stored and **how** entities relate to one another — independent of any physical storage details.

There is only **one conceptual schema** per database, acting as the single logical blueprint for the organisation.

Entities & Tables

STUDENT, COURSE, ENROLLMENT

Relationships

Foreign keys linking RollNo ➡ CourseID

Constraints & Types

Primary keys, data types, integrity rules

Internal Level (Physical Level)

The **Internal Level** is the lowest level of abstraction — closest to the hardware. It describes **how** data is physically stored, organised, and optimised for performance on disk.



File Organisation

Heap files, sorted files, or hashed files determine record arrangement on disk.



Indexing & Access Paths

B+ Trees, hash indexes, and clustered indexes speed up data retrieval dramatically.



Compression & Allocation

Data compression reduces storage costs; block allocation strategies optimise I/O performance.

Data Independence

A core objective of the three-level architecture is **Data Independence** — the ability to change one level without affecting the others. This is what makes modern databases flexible and maintainable.

Physical Data Independence

Changes at the **Internal Level** (e.g., switching from B+ Tree to hash indexing) do **not** affect the Conceptual Level's table structure.



Logical Data Independence

Changes at the **Conceptual Level** (e.g., adding a new column to a table) do **not** break existing External Level user views.



SIDE-BY-SIDE COMPARISON

Comparing the Three Levels

Aspect	External	Conceptual	Internal
Abstraction	Highest	Middle	Lowest
Audience	End users / Apps	DB designers / DBAs	System engineers
Focus	What users see	Logical structure	Physical storage
Schemas	Many (per user)	One per database	One per database
Example	Student grade view	ER diagram / tables	B+ Tree index files

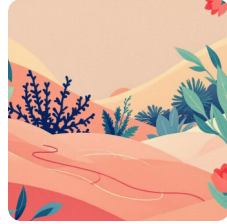
Each level insulates the others from change — that's the power of the architecture.

Advantages of the Three-Level Architecture



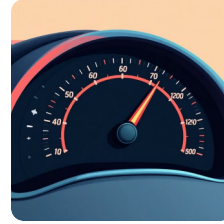
Security

Restrict data visibility per user role



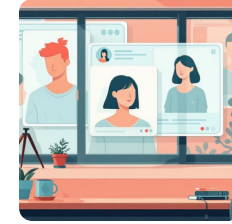
Flexibility

Modify storage or logic without breaking views



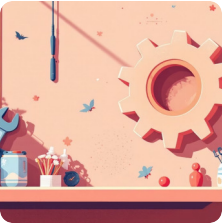
Performance

Tune physical storage independently



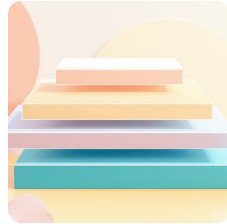
Multiple Views

Every stakeholder gets a tailored experience



Easy Maintenance

Reduced application rework when schema changes



Abstraction

Hide complexity, expose simplicity

Summary & Key Takeaways



External Level

What users see — customised, secure views



Conceptual Level

The logical blueprint — entities, relationships, constraints



Internal Level

How data lives on disk — files, indexes, blocks

This separation ensures **better security, flexibility, performance optimisation**, and **minimal impact of changes** across the entire database system.